

C'est quoi le problème d'un processus ???



Il est un peu lourd, cela implique :

- des recopies de mémoire
- et surtout de passer par le système pour communiquer avec d'autre processus.

Les threads

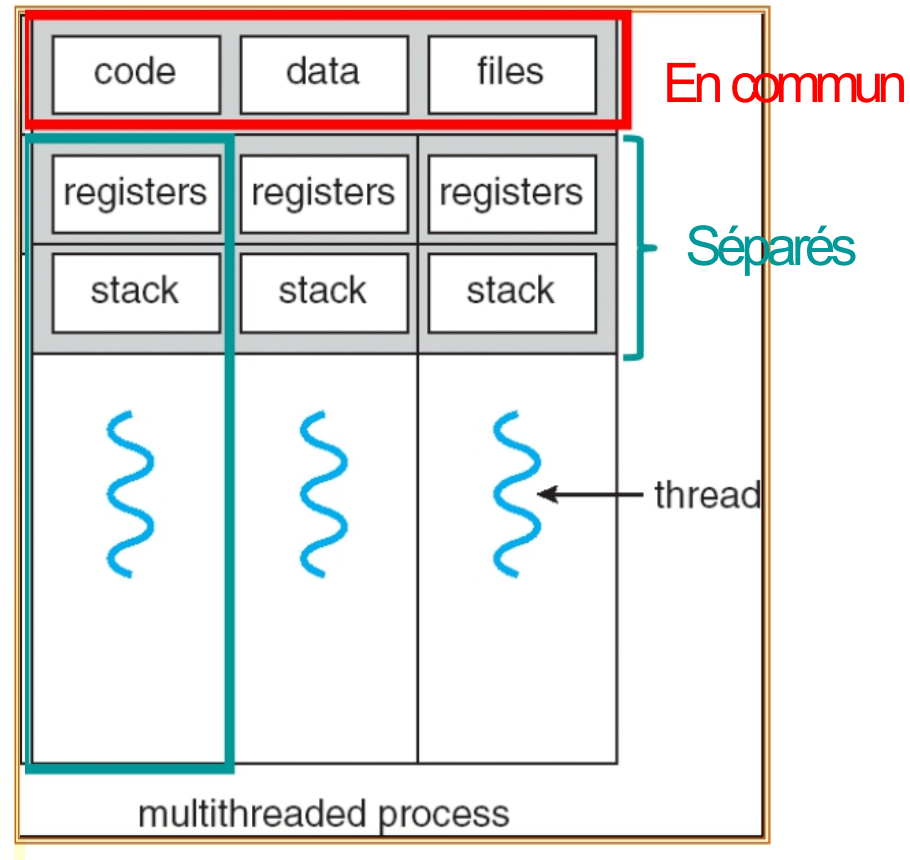
Concept de Thread

- Chaque processus peut fonctionner en lançant des sous-tâches internes au processus. Ces sous-tâches sont nommées, « flot d'exécution », « **processus légers** », ou « **Threads** ».
- De nombreux systèmes d'exploitation modernes offrent la possibilité d'associer à un même processus plusieurs chemins d'exécution (multithreading).
- Un thread, c'est un découpage d'un processus : Il utilise exactement la même mémoire qu'un processus, ainsi que les mêmes descripteurs de fichiers.

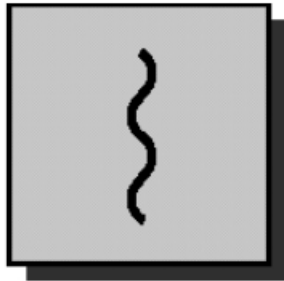
Concept de Thread

- Un **processus** contient tout ce qui est nécessaire pour l'exécution d'un programme (code, données(variables globales et statiques), pile, fichiers, périphériques...).
- Les threads **partagent certaines ressources** du processus mais ont aussi leurs propres ressources **privées**.
 - Ressources propres à chaque processus léger :
 - Les variables locales
 - Les registres
 - La pile d'exécution (stack)
 - Le compteur ordinal (Indique quelle instruction du code le thread exécute actuellement).
 - Ressources partagées entre les processus légers d'un même processus :
 - Le code
 - **Les variables globales**
 - La table des fichiers ouverts

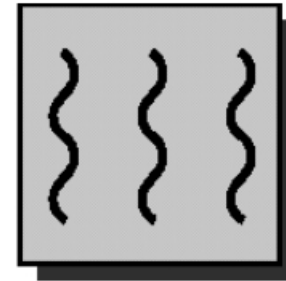
Concept de Thread



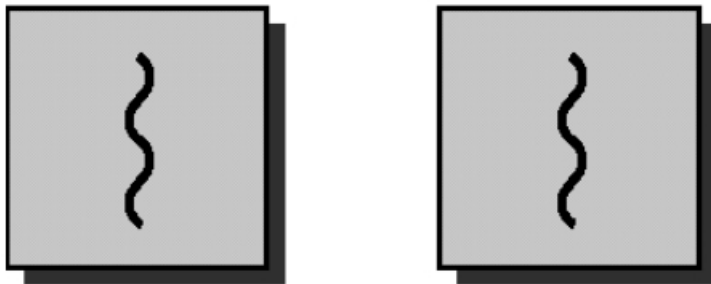
Threads et processus



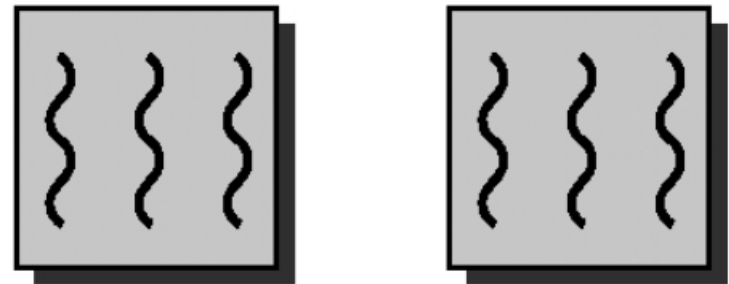
one process
one thread



one process
multiple threads



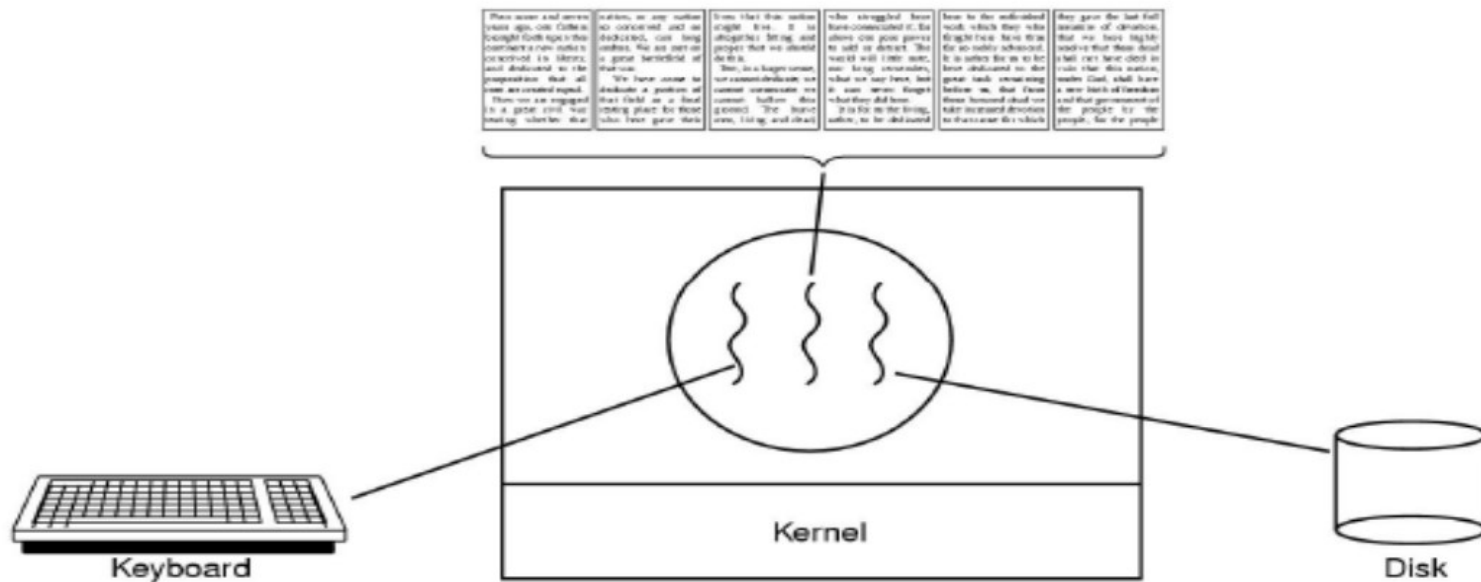
multiple processes
one thread per process



multiple processes
multiple threads per process

Cas d'utilisation de threads

Word processor



Le processus MS-Word peut impliquer plusieurs threads:

Interaction avec le clavier

Rangement de caractères sur la page

Sauvegarde régulière du travail fait

Contrôle d'orthographe

Etc.

Ces threads partagent tous le même fichier .doc et autres données

Avantages des Threads

- **Réactivité** : Un processus peut être subdivisé en plusieurs threads.
 - L'un peut s'exécuter tant que l'autre est bloqué.
- **Partage de ressources.**
- **Économie d'espace mémoire et de temps.**
 - Exemple : Sous Solaris, la création d'un processus est 30 fois plus lente que celle d'un Thread.
- **Moins de charge** pour créer, ordonnancer et détruire un thread.
- **Communication facile entre les Threads**

Types de threads

La majorité des systèmes permettent le multithreading (multiflot). Ils sont offerts soit au niveau utilisateur, soit au niveau noyau :

- Threads utilisateur

- Gérés par une bibliothèque utilisateur (par exemple **pthread** sous Linux ou **thread** dans Solaris)
- Le changement de contexte est rapide.

- Threads noyau

- Ils sont directement supportés par le noyau du système d'exploitation.
- Le système d'exploitation se charge de leur gestion

Service POSIX pour la gestion des threads

- Il définit un ensemble de structure de données et de fonctions C implantant des threads : *pthread*

`#include <pthread.h>`

- Création de threads

```
int pthread_create (pthread_t *thread , pthread_attr_t *attr,  
void *nomfonction, void *arg );
```

- Le service `pthread_create()` crée un processus léger qui exécute la fonction `nomfonction` avec l'argument `arg` et les attributs `attr`. (taille de pile, priorité, etc.) . Elle retourne zéro si le thread est créé sinon un nombre indiquant l'erreur.

Services POSIX pour la gestion des threads

- Suspension des threads

```
int pthread_join(pthread_t *thid, void **valeur_de_retour);
```

- pthread_join() suspend l'exécution d'un processus léger (appelant) jusqu'à ce que termine le processus léger avec l'identificateur *thid* et retourne son résultat dans *valeur_de_retour*. Cette fonction retourne l'état de terminaison du processus léger (zéro en cas de succès).

- Terminaison des threads

```
void pthread_exit(void *valeur_de_retour);
```

- pthread_exit() permet à un processus léger de terminer son exécution, en retournant l'état de terminaison.

Application 1

Écrire un programme C multi-threadé dans lequel le père déclare une variable globale ***nbr_parts_gateau*** initialisée à 8. Il lance par la suite 4 threads dont chacun cherche à consommer un part chaque seconde. Après chaque consommation, le fils affiche le nombre de parts restants.

Dans le cas où le fils ne trouve plus de morceaux, il affiche le message suivant : « thread x : Gâteau terminé !!! », où x est le numéro du thread correspondant.

NB: Le père doit attendre la terminaison de tous ses fils.

```

#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
volatile int nbr_parts_gateaux=8;
void* manger (void* args)
{
    while (nbr_parts_gateaux > 0)
    {
        nbr_parts_gateaux--;
        printf ("thread%d : nombre de parts restants = %d\n", (int)(long)args, nbr_parts_gateaux);
        sleep(1);
    }
    printf ("thread%d : Gateau terminé\n", (int)(long)args);
    return 0;
}
int main()
{
    pthread_t tab[4];
    int i;
    for (i=0; i<4; i++)
        pthread_create(&tab[i], NULL, manger, (void*)(long) i);
    for (i=0; i<4; i++)
        pthread_join(tab[i], NULL); /*Eviter le risque de terminer le programme complètement
                                     sans avoir pu exécuter les threads */
    return 0;
}

```

La valeur sera toujours relue en mémoire à chaque accès et non stockée dans un registre temporaire

NULL pour utiliser taille de pile et priorité par défaut

NULL : pas besoin de retour du thread appelé

/*Eviter le risque de terminer le programme complètement sans avoir pu exécuter les threads */

Application 2

Écrire un programme C multi-threadé dans lequel le père déclare une variable globale **litres_eau** initialisée à 10. Il lance par la suite 3 threads, chacun représentant une pompe qui cherche à retirer un litre d'eau toutes les 2 secondes. Après chaque retrait, le thread affiche le nombre de litres restants dans le réservoir.

Dans le cas où le réservoir ne contient plus d'eau, le thread affiche le message suivant :

Pompe x : Réservoir vide !!!

où x est le numéro du thread correspondant.

NB: Le père doit attendre la terminaison de tous ses fils avant de terminer le programme.